

Paso 12 - Sobrecarga | Solucionario

Las cinco soluciones del paso 12. Todas viven del mismo principio: **el nombre no identifica un método, lo identifican el nombre y sus parámetros**.

Úsalo después de intentarlo. Cada solución viene con la salida real del programa: si la tuya coincide, está bien aunque el código sea distinto. Los `.java` de esta carpeta son los mismos que se muestran abajo, listos para ejecutar: `cd soluciones/paso12` y `javac *.java`.

B12.1 - perimetro, igual que area

Tres versiones más, con las mismas firmas que `area`.

▼ Ver la solución (Geometria.java, B12_1_Perimetro.java)

Geometria.java

```
/*
 * GEOMETRIA - la del paso 12, con el perimetro (B12.1) y esTriangulo (B12.2).
 */
public class Geometria {

    public static double area(double radio) {
        return Math.PI * radio * radio;
    }

    public static double area(double base, double altura) {
        return base * altura;
    }

    public static double area(double a, double b, double c) {
        double s = (a + b + c) / 2;
        return Math.sqrt(s * (s - a) * (s - b) * (s - c));
    }

    // ----- B12.1 -----

    public static double perimetro(double radio) {
        return 2 * Math.PI * radio;
    }

    public static double perimetro(double base, double altura) {
        return 2 * (base + altura);
    }

    public static double perimetro(double a, double b, double c) {
        return a + b + c;
    }

    // ----- B12.2 -----

    public static boolean esTriangulo(double a, double b, double c) {
        return a < b + c && b < a + c && c < a + b;
    }
}
```

B12_1_Perimetro.java

```
public class B12_1_Perimetro {
    public static void main(String[] args) {
        String formato = "%-26sarea %6.2f    perimetro %6.2f%n";

        System.out.printf(formato, "Circulo de radio 2:",
            Geometria.area(2), Geometria.perimetro(2));
        System.out.printf(formato, "Rectangulo de 3 x 4:",
            Geometria.area(3, 4), Geometria.perimetro(3, 4));
        System.out.printf(formato, "Triangulo de lados 3-4-5:",
            Geometria.area(3, 4, 5), Geometria.perimetro(3, 4, 5));
    }
}
```

Salida:

```
Circulo de radio 2:      area  12.57    perimetro  12.57
Rectangulo de 3 x 4:     area  12.00    perimetro  14.00
Triangulo de lados 3-4-5: area   6.00    perimetro  12.00
```

Lo que importa. `area` y `perimetro` tienen **tres versiones cada una** y Java no se confunde nunca, porque para él un método son su nombre **y** sus parámetros. Seis métodos, dos nombres.

Guardar el molde en una variable (`String formato = ...`) evita repetirlo tres veces: si quieres cambiar el ancho de la columna, tocas una línea. Es la misma idea de no duplicar del B9.3.

Lo del círculo de radio 2 no es un error: su área es $\pi r^2 = 4\pi$ y su perímetro $2\pi r = 4\pi$. El único radio donde coinciden.

B12.2 - Validar el triángulo

El paso 11 aplicado al paso 12: preguntar **antes** de calcular.

▼ Ver la solución (B12_2_Triangulo.java)

B12_2_Triangulo.java

```
public class B12_2_Triangulo {
    public static void main(String[] args) {
        mostrar(3, 4, 5);
        mostrar(1, 1, 5);
        mostrar(2, 2, 4);    // el caso justo: 4 = 2 + 2, no es triangulo

        System.out.println();
        System.out.println("Sin validar, area(1, 1, 5) = " + Geometria.area(1,
1, 5));
    }

    static void mostrar(double a, double b, double c) {
        if (Geometria.esTriangulo(a, b, c)) {
            System.out.printf("Lados %.0f, %.0f, %.0f:  area %.2f%n", a, b, c,
Geometria.area(a, b, c));
        } else {
            System.out.printf("Lados %.0f, %.0f, %.0f:  ese triangulo no
existe%n", a, b, c);
        }
    }
}
```

Salida:

```
Lados 3, 4, 5:  area 6.00
Lados 1, 1, 5:  ese triangulo no existe
Lados 2, 2, 4:  ese triangulo no existe

Sin validar, area(1, 1, 5) = NaN
```

Qué es NaN. *Not a Number*: el resultado de una operación decimal imposible, aquí la raíz de un número negativo. Lo peor que tiene es que **no revienta**: no hay excepción, no hay aviso, y si lo sumas a algo, el resultado también es NaN. Un dato corrupto que se propaga en silencio.

El caso 2, 2, 4 es el borde: 4 es exactamente 2 + 2, así que el triángulo está "aplastado" y no existe. Por eso la condición usa < y no <=. Si hubieras escrito <=, este caso pasaría la validación y el área daría 0.0, que parece un número válido.

Fíjate en que `mostrar` es `static` y está en el programa, no en `Geometria`: `Geometria` calcula y responde, no imprime. La misma división de trabajo del B11.2.

B12.3 - El promedio ponderado

Una segunda versión de `promedio`, con un arreglo más.

▼ Ver la solución (Notas.java, B12_3_Ponderado.java)

Notas.java

```
/*
 * NOTAS - con la segunda version de promedio (B12.3).
 */
public class Notas {

    public static final double NOTA_MINIMA = 1.0;
    public static final double NOTA_MAXIMA = 7.0;
    public static final double NOTA_APROBACION = 4.0;

    public static double promedio(double[] notas) {
        double suma = 0;
        for (double n : notas) {
            suma += n;
        }
        return suma / notas.length;
    }

    // B12.3: misma idea, pero cada nota pesa distinto
    public static double promedio(double[] notas, double[] pesos) {
        double suma = 0;
        for (int i = 0; i < notas.length; i++) {
            suma += notas[i] * pesos[i];
        }
        return suma;
    }

    public static boolean aprobo(double promedio) {
        return promedio >= NOTA_APROBACION;
    }
}
```

B12_3_Ponderado.java

```

public class B12_3_Ponderado {
    public static void main(String[] args) {
        double[] notas = {5.0, 6.0, 4.0};
        double[] pesos = {0.3, 0.3, 0.4};

        System.out.printf("Promedio simple:    %.1f%n", Notas.promedio(notas));
        System.out.printf("Promedio ponderado: %.1f%n", Notas.promedio(notas,
        pesos));

        // La misma nota mas alta pesando 40% en vez de 30%
        double[] otrosPesos = {0.3, 0.4, 0.3};
        System.out.printf("Con otros pesos:    %.1f%n", Notas.promedio(notas,
        otrosPesos));
    }
}

```

Salida:

```

Promedio simple:    5.0
Promedio ponderado: 4.9
Con otros pesos:    5.1

```

Lo que importa. El ponderado **no divide**: los pesos ya suman 1.0, así que la suma de `nota * peso` ya es el promedio. Si dividieras por 3 como en el simple, darías 1.63.

Y necesita un `for` con índice, no un for-each: hay que recorrer **dos** arreglos en paralelo, tomando `notas[i]` junto con `pesos[i]`. El for-each solo sabe recorrer uno.

Las respuestas de las preguntas. Si los pesos no suman 1.0, el resultado no es un promedio: con pesos 0.5, 0.5, 0.5 te puede dar 7.5, una nota que no existe. Y si el arreglo de pesos es más corto, `pesos[i]` revienta con `ArrayIndexOutOfBoundsException` (paso 8). Las dos son entradas malas que el método acepta sin chistar: la solución de verdad es validarlas, y desde la demo 10, lanzar una excepción cuando no se cumplen.

B12.4 - sumar para todo

Cuatro versiones y cinco llamadas, para ver cuál elige Java en cada una.

▼ Ver la solución (Calculos.java, B12_4_ProbarCalculos.java)

Calculos.java

```
public class Calculos {

    public static int sumar(int a, int b) {
        System.out.print(" [uso (int, int)] ");
        return a + b;
    }

    public static double sumar(double a, double b) {
        System.out.print(" [uso (double, double)] ");
        return a + b;
    }

    public static int sumar(int a, int b, int c) {
        System.out.print(" [uso (int, int, int)] ");
        return a + b + c;
    }

    public static int sumar(int[] numeros) {
        System.out.print(" [uso (int[])] ");
        int suma = 0;
        for (int n : numeros) {
            suma += n;
        }
        return suma;
    }
}
```

B12_4_ProbarCalculos.java

```

public class B12_4_ProbarCalculos {
    public static void main(String[] args) {
        System.out.println(Calculos.sumar(2, 3));
        System.out.println(Calculos.sumar(2.5, 3.5));
        System.out.println(Calculos.sumar(1, 2, 3));
        System.out.println(Calculos.sumar(new int[] {1, 2, 3, 4}));
        System.out.println(Calculos.sumar(2, 3.5));
    }
}

```

Salida:

```

[uso (int, int)]          5
[uso (double, double)]   6.0
[uso (int, int, int)]     6
[uso (int[])]            10
[uso (double, double)]   5.5

```

La respuesta: `sumar(2, 3.5)` usa la versión `(double, double)`. No calza exacto con ninguna, así que Java busca la que **pueda** recibir esos datos convirtiendo lo menos posible: un `int` cabe en un `double` sin perder nada (es una *conversión ensanchadora*), así que el 2 se convierte en 2.0. Al revés no lo haría nunca: convertir 3.5 a `int` perdería el .5, y eso Java no lo hace solo.

Si borras la versión `(double, double)`, no compila:

```

B12_4_ProbarCalculos.java:7: error: incompatible types: possible lossy conversion
from double to int
    System.out.println(Calculos.sumar(2, 3.5));
                                   ^
Note: Some messages have been simplified; recompile with -Xdiags:verbose to get full
output

```

La única candidata que queda es `(int, int)`, y para usarla habría que convertir 3.5 a `int` perdiendo el .5. Java prefiere **no compilar** antes que redondear a tus espaldas. Ese **Note:** del final es un dato útil para todo el curso: con `javac -Xdiags:verbose` el compilador te muestra **todas** las versiones que consideró y por qué descartó cada una.

El truco de imprimir `[uso (...)]` dentro de cada versión es una técnica de depuración que sirve toda la vida: cuando no sabes qué código se está ejecutando, ponle un mensaje.

B12.5 - Desafío: la tarjeta de saludo

Tres versiones encadenadas: las dos cortas solo rellenan lo que falta y llaman a la larga, que es la única que dibuja.

▼ Ver la solución (B12_5_Tarjeta.java)

B12_5_Tarjeta.java

```
public class B12_5_Tarjeta {

    static void tarjeta(String nombre) {
        tarjeta(nombre, "Feliz cumpleaños");
    }

    static void tarjeta(String nombre, String mensaje) {
        tarjeta(nombre, mensaje, '*');
    }

    static void tarjeta(String nombre, String mensaje, char borde) {
        int ancho = Math.max(mensaje.length(), nombre.length()) + 4;

        dibujarLinea(ancho, borde);
        System.out.printf("%c %- " + (ancho - 4) + "s %c%n", borde, mensaje,
borde);
        System.out.printf("%c %- " + (ancho - 4) + "s %c%n", borde, nombre,
borde);
        dibujarLinea(ancho, borde);
        System.out.println();
    }

    static void dibujarLinea(int ancho, char borde) {
        for (int i = 0; i < ancho; i++) {
            System.out.print(borde);
        }
        System.out.println();
    }

    public static void main(String[] args) {
        tarjeta("Ana");
        tarjeta("Luis", "Que te mejores");
        tarjeta("Sofia", "Feliz titulacion", '#');
    }
}
```

Salida:

```

*****

* Feliz cumpleaños *
* Ana                *
*****

*****

* Que te mejores *
* Luis            *
*****

#####
# Feliz titulacion #
# Sofia          #
#####

```

Lo que importa. El dibujo está escrito **una sola vez**. Las otras dos versiones son una línea cada una. Si mañana cambias el diseño del recuadro, hay un solo lugar donde tocarlo.

En Python esto sería una función:

```

def tarjeta(nombre, mensaje="Feliz cumpleaños", borde="*"):
    ...

```

Una función con valores por defecto contra tres métodos sobrecargados. Python es más corto; Java es más explícito (cada versión aparece en la documentación con su firma). Es el mismo contraste de la demo 1 con los constructores, y la tabla del README del curso lo dice en una línea: **Java resuelve con muchas versiones, Python con una flexible.**

El `"%c %- " + (ancho - 4) + "s %c%n"` arma el molde del `printf` **concatenando**, porque el ancho se calcula en tiempo de ejecución. Es un truco útil, y también la razón por la que un `printf` mal armado revienta recién al ejecutar (paso 2).

Volver a la [hoja del paso 12](#) o al [paso 12 de la guía](#).